

Optimizing Edge AI: Integration of CMSIS-NN Kernels into the ZANT Cross-Compilation Framework

Vincenzo Del Grosso
*Department of Electronics,
Information and Bioengineering*
Politecnico di Milano,
vincenzo.delgrosso@mail.polimi.it

Emanuele Severino
*Department of Electronics,
Information and Bioengineering*
Politecnico di Milano,
emanuele.severino@mail.polimi.it

Riccardo Scarabelli
*Department of Electronics,
Information and Bioengineering*
Politecnico di Milano,
riccardo.scarabelli@mail.polimi.it

Abstract—The proliferation of Internet of Things (IoT) devices has driven a paradigm shift from cloud-centric computing to Edge AI, where machine learning inference occurs directly on local hardware. This transition offers significant benefits regarding latency, bandwidth, and privacy, but it presents substantial challenges due to the severe resource constraints of microcontrollers (MCUs). Existing solutions for deploying Neural Networks (NN) on embedded systems often suffer from a lack of flexibility, heavy dependencies, or complex manual optimization. This paper presents the development and optimization of ZANT, an open-source Software Development Kit (SDK) designed to simplify the deployment of NN models on edge devices. ZANT leverages the Zig programming language to cross-compile ONNX models into optimized, dependency-free static libraries compatible with C. The specific focus of this work is the technical integration of ARM’s CMSIS-NN library into the ZANT pipeline. Integrating a complex, dependency-heavy C library into a modern, streamlined Zig framework represents a significant engineering challenge, necessitating a bridge between high-level model design and low-level hardware instructions. The primary objective was to replace generic mathematical kernels with hardware-accelerated DSP instructions to reduce inference latency. However, experimental results show significant variability. Benchmarks conducted on Cortex-M platforms indicate that while the integration was successful, the performance gains are not uniform across all kernel types. The results demonstrate that ZANT’s native, lightweight implementation often rivals or even outperforms the hardware-specific optimizations in certain configurations. This suggests that the theoretical gains of DSP acceleration are not absolute and must be carefully weighed against data movement costs and memory bandwidth constraints, which can dominate execution time on constrained architectures.

Index Terms—Edge AI, TinyML, CMSIS-NN, ONNX, Embedded Systems, ZANT Framework

I. INTRODUCTION

The rapid evolution of Information and Communication Technology (ICT) has permeated every aspect of modern life, leading to the era of ubiquitous sensing and computing. As the number of connected devices grows, the traditional cloud-computing paradigm is becoming increasingly unsustainable due to latency requirements and bandwidth limitations. This has given rise to Edge AI, or TinyML, a discipline focused

on executing machine learning inference directly on end-point devices such as microcontrollers (MCUs).

However, the transition of deep learning models from servers to embedded devices is full of challenges. MCUs, such as the ARM Cortex-M series, operate under extreme resource constraints, often possessing limited RAM and lacking GPUs. Furthermore, the software ecosystem is fragmented; standard frameworks like PyTorch are too heavy, and developers face the “deployment gap”: the difficulty of translating a trained Neural Network (NN) into efficient, executable C code. In Convolutional Neural Networks (CNNs), the convolution kernels account for the majority of the total execution time. Therefore, the prevailing hypothesis in embedded engineering is that achieving viable inference speeds requires utilizing hardware-specific optimizations, such as Single Instruction, Multiple Data (SIMD) and Digital Signal Processing (DSP) instructions.

This paper focuses on the ZANT SDK, an innovative open-source tool written in the Zig programming language. ZANT acts as a bridge between high-level ONNX models and embedded firmware, leveraging Zig’s cross-compilation capabilities to generate dependency-free static libraries [1]. The central research theme of this work is the integration of the CMSIS-NN library into ZANT. The goal was to verify if replacing ZANT’s “Lean” generic kernels with ARM’s highly optimized CMSIS-NN kernels would universally drastically reduce inference latency. This integration is non-trivial. It involves embedding a legacy C-based library, rich in dependencies and macros, into the modern, memory-safe architecture of a Zig framework. The process requires a careful mapping of ONNX operators to CMSIS-NN functions and complex memory management to bridge the two languages.

While the engineering phase of integrating CMSIS-NN was successfully completed, the subsequent benchmarking phase yielded unexpected results. Contrary to the assumption that hardware-specific kernels are universally superior, our tests show no clear linear pattern of improvement. In several instances, the strict data layout requirements of the hardware kernels or the memory-efficient nature of ZANT’s native

Zig code resulted in the generic implementation performing comparably to, or better than, the CMSIS-NN accelerated version. Consequently, this paper does not merely present an optimization method, but rather a critical analysis of the trade-offs between "bare-metal" generic implementations and vendor-specific hardware acceleration.

The main contributions of this work are the following:

- A detailed analysis of the ZANT framework architecture and its handling of ONNX Intermediate Representations.
- The methodology for integrating the CMSIS-NN hardware abstraction layer (C-based) into a Zig-based cross-compilation pipeline, detailing the interoperability challenges.
- The implementation of operator mapping strategies that dynamically select between generic ZANT kernels and optimized CMSIS-NN kernels.
- A comparative performance evaluation on ARM-based platforms, highlighting that hardware acceleration provides selective rather than universal benefits compared to lightweight generic code.

The remainder of the paper is organized as follows: Section II discusses related works. Section III describes the proposed method and the integration challenges. Section IV reports the experimental results, analyzing the specific cases where CMSIS-NN succeeds or fails to improve latency. Finally, Section V concludes the paper.

II. RELATED WORKS

The domain of embedded machine learning requires a rigorous analysis of the software boundaries between hardware capabilities and application logic. This section identifies the established standards in the Arm ecosystem and analyzes the specific methods used in CMSIS-NN.

A. The CMSIS Ecosystem Boundaries

To understand the execution environment of modern MCUs, one must look at the abstraction layers provided by hardware vendors. The primary boundary is defined by the Common Microcontroller Software Interface Standard (CMSIS), which acts as a bridge between hardware and application. This ecosystem solves hardware fragmentation but does not inherently address the computational density required for Deep Learning (DL) inference.

B. CMSIS-NN: Kernel-Level Optimization

The most relevant research to this work is **CMSIS-NN**, a library of hand-optimized neural network kernels designed to maximize performance on resource-constrained devices. It addresses the limitation that generic C implementations of mathematical operators are often too slow for real-time inference [2]–[6].

1) *Optimization Methods*: CMSIS-NN employs specific algorithmic optimizations:

- **SIMD (Single Instruction Multiple Data)**: On Cortex-M4/M7, it utilizes SIMD to perform multiple arithmetic operations in a single clock cycle.

- **MVE (M-Profile Vector Extension)**: On newer architectures (Cortex-M55/M85), it leverages Helium technology for vectorized operations.
- **Quantization Support**: It operates on `int8` and `int16` data types, adhering to TFLM specifications to reduce memory bandwidth.

2) *Functional Scope*: The library focuses on compute-heavy operations like `Conv2D` and `DepthwiseConv2D`. By utilizing pre-processor macros, the library selects the code path at compile time. However, this heavy reliance on macros and strictly typed C structures poses integration challenges when interfacing with modern, flexible languages like Zig.

C. Gap Analysis and Value Curve

While CMSIS-NN represents the state-of-the-art for *kernel-level* optimization on Arm, it has limitations regarding system-level innovation.

1) *Pros of Existing Solutions*:

- **Hardware Efficiency**: CMSIS-NN achieves near-theoretical peak performance for specific mathematical operations.
- **Standardization**: It creates a unified optimization backend for frameworks like TensorFlow Lite Micro.

2) *Cons and Identified Problem Space*: The analysis identifies boundaries where CMSIS-NN falls short, creating the opportunity for ZANT:

- 1) **Scope Limitation**: CMSIS-NN is purely a mathematical library without graph parsing or scheduling capabilities.
- 2) **Architecture Lock-in**: It is tightly coupled to Arm architecture.
- 3) **Integration Rigidity**: Being a static C library, integrating it into non-C frameworks involves significant overhead in data marshalling and ABI compatibility.

3) *Positioning the Proposed Innovation (ZANT)*: The proposed ZANT SDK elevates the abstraction level. ZANT functions as an end-to-end compiler rather than just a calculator. The hypothesis driving this work was that embedding CMSIS-NN into ZANT would combine ZANT's flexibility with ARM's speed. However, as discussed in the following sections, the "Gap Analysis" reveals that the performance gap between efficient generic code (ZANT) and hardware-specific code (CMSIS-NN) is narrower than expected in real-world inference scenarios, challenging the assumption that vendor libraries are always the optimal choice.

III. METHOD

The proposed solution, ZANT, acts as a bridge between high-level neural network definitions (e.g., ONNX) and bare-metal execution on resource-constrained microcontrollers. The methodology adopted relies on a hybrid architecture that leverages the safety and metaprogramming capabilities of the Zig programming language while integrating the industry-standard performance of the CMSIS-NN C library. This section details the integration strategy and the optimization pipeline.

A. Hybrid Build System and CMSIS-NN Integration

A critical challenge in embedded machine learning is the friction between high-level model compilers and low-level hardware dependencies. To resolve this, we developed a unified build system based on the Zig build infrastructure. Unlike traditional approaches that link against pre-compiled static libraries, the ZANT method compiles CMSIS-NN kernels from source alongside the main application. This approach ensures ABI (Application Binary Interface) compatibility and allows for fine-grained control over compilation flags.

1) *Unified Compilation Orchestration*: The integration is managed through a custom build logic (defined in `build.zig` and `build_utils.zig`) that eliminates the need for external Makefiles or CMake scripts during the deployment phase. The system detects the target architecture (e.g., `thumb-free-standing-eabi`) thanks to a flag at compile time and configures the environment for cross-compilation.

The build process follows a specific sequence to integrate the CMSIS-NN C artifacts into the Zig module:

- 1) **Path Resolution**: The system creates a module structure that resolves the paths to CMSIS-Core, CMSIS-DSP, and CMSIS-NN headers. It handles the variance in toolchain locations across different operating systems (Linux, Windows, MacOS) by probing standard directories for Arm GNU Toolchain headers.
- 2) **Selective Kernel Compilation**: Rather than compiling the entire CMSIS-NN library, the build system selectively compiles only the required source files. As seen in the implementation, specific kernels such as `arm_convolve_s8.c`, `arm_depthwise_conv_s8.c`, and support functions are added via `addCSourceFile`. This reduces the compilation time and binary load.
- 3) **Flag Injection and Optimization**: To match the performance of native C implementations, the build system injects hardware-specific flags directly into the C compilation step.
 - `-Ofast`: Enforces aggressive optimization for the kernels.
 - `-fno-math-errno`: Disables standard math error handling to speed up floating-point operations where applicable.
 - `-DOPTIONAL_RESTRICT_KEYWORD=__restrict`: Enables pointer aliasing optimizations critical for Cortex-M7 and newer architectures.

B. 8-bit Quantization Strategy

Given the severe memory and compute constraints of microcontroller-class devices, the proposed methodology adopts 8-bit integer quantization for neural network inference. Quantization reduces both the memory footprint and computational complexity of neural network models by mapping floating-point values to low-precision integer representations. In particular, 8-bit quantization has emerged as a de facto standard for efficient inference on embedded and edge platforms

[7]–[14]. For these reasons, the ZANT convolution function that will be considered will be the `qLinearConv` function, which performs the convolution quantized on 8 bits, while the CMSIS-NN functions considered will be those quantized in `int s8`

1) *C-Zig Interoperability Layer*: The method utilizes Zig’s native C import capabilities (`@cImport`) to expose the CMSIS functions to the ZANT codebase. The build system configures the `configureCmsisSupport` routine to ensure that the generated Zig object files can link against the C object files without symbol conflicts.

Furthermore, the system is designed to be modular. A global feature flag (`enable_cmsis`) controls the dependency injection. When enabled, the build system automatically links ‘LibC’ and ensures that the generic ZANT operators are replaced or augmented by the CMSIS-NN equivalents. This allows for rapid testing between the generic ZANT implementation and the CMSIS-optimized version within the same testing framework.

C. Runtime Operator Orchestration and Interoperability

The core innovation in the ZANT execution engine is its ability to map high-level tensor operations to the optimal underlying implementation without user intervention. This pipeline consists of three distinct stages: Static Dispatch, Data Layout Adaptation, and Kernel Execution.

1) *Compile-Time Static Dispatch*: The entry point for inference is the high-level `qlinearconv` operator defined in the ZANT core. To minimize runtime overhead, the decision to use cmsis-nn implementation or the native one is made at compile-time using Zig’s `comptime` features.

This mechanism ensures that the final binary only contains the necessary code paths, significantly reducing the flash memory footprint.

2) *The Zero-Copy Interoperability Wrapper*: The bridge between ZANT and the CMSIS-NN C library is handled by a specialized Zig wrapper. This layer addresses critical data mismatches, in particular in tensor’s format, between the ONNX specification (which often uses `uint8` and `[N, C, H, W]` layouts) and CMSIS-NN (which strictly requires `int8` and specific memory ordering). The wrapper also prepares the parameters to be passed as argument to the cmsis-nn functions using the data structures provided by the cmsis library.

An important detail in terms of flexibility is the fact that the function that the wrapper exposes for the convolution call has the same signature as that of the native function, so that calling one or the other is completely transparent at the intermediate representation and codegen levels.

To maintain high performance, the wrapper employs a “Zero-Copy” strategy whenever possible:

- 1) **Arena Allocation Strategy**: To prevent heap fragmentation—a common issue in long-running embedded systems—the wrapper initializes a temporary `ArenaAllocator` for all scratch memory required during the convolution layer’s execution. This ensures

deterministic memory usage and single-step deallocation.

- 2) **Smart Weight Packing:** The wrapper analyzes the weight tensor at runtime. If the weights are already compatible (Type `int8` and symmetric quantization), the pointer is passed directly to the kernel.
- 3) **Vectorized Type Adaptation:** CMSIS-NN operates exclusively on `int8` so a conversion from unsigned (`u8`) to signed (`int8`) type is necessary for the correct execution of CMSIS-NN convolution.

3) *Low-Level Kernel Integration:* The final stage delegates the heavy computational lifting to the CMSIS-NN C kernels. The C implementation (`arm_convolve_wrapper_s8.c`, `arm_depthwise_conv_wrapper_s8.c`) acts as a secondary dispatcher, selecting the most efficient algorithm—such as 1×1 direct convolution or `Im2Col`-based GEMM—based on the layer’s geometric properties (stride, padding, and dilation).

IV. EXPERIMENTAL RESULTS AND DISCUSSION

This section evaluates the effectiveness of the proposed ZANT execution engine by comparing it against the industry-standard CMSIS-NN kernels. The analysis focuses on execution latency across a variety of Convolutional Neural Network (CNN) layer configurations, ranging from standard 3×3 kernels to pointwise and depthwise operations.

A. Experimental Setup

All experiments were conducted on two distinct hardware platforms to cover different performance tiers: the **Arduino Nicla Vision** (STM32H747XI, dual-core, Cortex-M7 at 480 MHz) and the **Nucleo-64** (STM32L476RGT6, Cortex-M4 at 80 MHz).

1) *Measurement Protocol:* To ensure statistical significance and reproducibility:

- **Input Data and Determinism:** Since the evaluated CMSIS-NN kernels exhibit data-independent control flow (i.e., no sparse optimizations), input tensors were primarily initialized with deterministic fixed values. To validate this, comparative tests with random inputs were conducted, confirming no statistically significant deviation in execution time.
- **Data Type Consistency:** Experiments compared standard `int8` execution flows. Where ZANT utilized `uint8`, an optimized SIMD-based conversion routine was employed. Preliminary validation confirmed that the overhead of this conversion is negligible compared to the kernel execution time.
- **Timing Mechanism:** Execution time was measured in microseconds (μs) using the high-resolution hardware timers available on the MCUs.
- **Compilation:** ZANT was compiled with `ReleaseSmall` optimizations via the hybrid build system. CMSIS-NN kernels were linked with standard DSP flags enabled (e.g., `-Ofast`, `-DARM_MATH_DSP`).

B. Comparative Analysis

We evaluated distinct layer configurations to stress different aspects of the execution engine: SIMD utilization, memory bandwidth, and kernel loop overhead. The results are summarized in Table I and Table II.

TABLE I
NICLA VISION: INFERENCE LATENCY COMPARISON (LOWER IS BETTER)

ID	Kernel	Input / Channels	CMSIS (μs)	ZANT (μs)	Δ %
1	3×3	$16 \times 16 / 3 \rightarrow 6$	1,707	1,908	+11.7%
2	3×3	$100 \times 100 / 3 \rightarrow 6$	63,120	74,239	+17.6%
3	3×3	$50 \times 50 / 16 \rightarrow 16$	149,262	153,766	+3.0%
4	3×3	$16 \times 16 / 32 \rightarrow 32$	56,655	51,404	-9.3%
5	5×5	$16 \times 16 / 3 \rightarrow 6$	4,369	3,735	-14.5%
6	5×5	$100 \times 100 / 3 \rightarrow 6$	165,117	148,265	-10.2%
7	5×5	$50 \times 50 / 16 \rightarrow 16$	406,229	362,946	-10.6%
8	5×5	$16 \times 16 / 32 \rightarrow 32$	158,234	116,272	-26.5%
9	PW 1×1	$16 \times 16 / 32 \rightarrow 32$	5,695	5,348	-6.1%
10	DW 3×3	$32 \times 32 / 32 \rightarrow 32$	12,878	19,582	+52.0%
11	DW 5×5	$32 \times 32 / 32 \rightarrow 32$	77,686	32,762	-57.8%
12	5×5	$32 \times 32 / 3 \rightarrow 32$	66,895	80,174	+19.9%
13	5×5	$16 \times 16 / 32 \rightarrow 32$	158,107	116,305	-26.4%
14	5×5	$8 \times 8 / 32 \rightarrow 64$	77,355	52,578	-32.0%

TABLE II
NUCLEO-64: INFERENCE LATENCY COMPARISON (LOWER IS BETTER)

ID	Kernel	Input / Channels	CMSIS (μs)	ZANT (μs)	Δ %
1	3×3	$16 \times 16 / 3 \rightarrow 16$	31,086	39,801	+21.90%
2	3×3	$32 \times 32 / 4 \rightarrow 16$	170,684	201,585	+15.33%
3	3×3	$16 \times 16 / 16 \rightarrow 16$	154,823	129,976	-19.11%
4	3×3	$24 \times 24 / 16 \rightarrow 16$	359,254	402,342	+10.70%
5	5×5	$16 \times 16 / 3 \rightarrow 16$	39,777	29,487	-34.89%
6	5×5	$32 \times 32 / 4 \rightarrow 16$	444,459	421,977	-5.32%
7	5×5	$16 \times 16 / 16 \rightarrow 16$	422,189	343,660	-22.85%
8	5×5	$24 \times 24 / 16 \rightarrow 16$	982,876	916,547	-7.23%
9	DW 3×3	$24 \times 24 / 4 \rightarrow 16$	15,296	45,098	+66.08%
10	DW 3×3	$16 \times 16 / 16 \rightarrow 16$	16,053	19,997	+19.72%
11	DW 3×3	$16 \times 16 / 16 \rightarrow 32$	28,679	39,981	+28.27%
12	DW 5×5	$24 \times 24 / 4 \rightarrow 16$	25,682	75,360	+65.92%
13	DW 5×5	$16 \times 16 / 16 \rightarrow 16$	93,534	32,946	-183.90%
14	DW 5×5	$16 \times 16 / 16 \rightarrow 32$	45,818	65,858	+30.43%
15	PW 1×1	$24 \times 24 / 4 \rightarrow 16$	12,603	28,323	+55.50%
16	PW 1×1	$16 \times 16 / 32 \rightarrow 32$	57,944	45,621	-27.01%

C. Discussion

The results reveal a nuanced performance landscape. While the integration of CMSIS-NN was successful, the benchmark data indicates that hardware acceleration via CMSIS-NN does not strictly guarantee lower latency compared to ZANT’s native implementation. This counter-intuitive outcome requires a deeper analysis of the underlying architectural factors, particularly regarding memory traffic and build system configuration.

1) *The Main Reasons Behind Speed-ups:* Looking at the benchmark results, it’s clear that the performance achieved with a given input parameters configuration can differ significantly from that achieved with another one, even if very similar. This is due to the way both the native and CMSIS

versions have implementations dedicated to specific input parameter configurations, the main ones being: 3x3 filters, `input_channels = output_channel`, number of input channels, depthwise convolution, pointwise convolution, stride size, etc.

2) *The Memory Bandwidth Hypothesis (im2col and Cache Thrashing)*: A significant observation is ZANT's unexpected superiority in larger kernels (e.g., 5×5 cases), where it outperforms CMSIS-NN by up to 26%. This is likely not due to wrapper overhead—which utilizes vectorized operations—but rather to the algorithmic approach of CMSIS-NN itself. The library extensively uses the `im2col` (image-to-column) transformation to map convolutions to matrix multiplications (GEMM). While `im2col` optimizes computational throughput, it significantly increases memory traffic and buffer usage. On MCUs with limited cache lines (or no data cache on the M4), this memory expansion can lead to cache thrashing. The CPU stalls waiting for data fetch, negating the speed advantage of DSP instructions. In contrast, ZANT's native implementation likely employs a more direct, loop-based approach that, while computationally less dense, is more "cache-friendly" and minimizes memory bandwidth pressure.

3) *Compiler Optimization and Flag Activation*: Another potential cause for the performance disparity lies in the complexity of the hybrid build system. Integrating a massive C library like CMSIS-NN into a Zig-based pipeline is non-trivial. While the build system was configured to pass standard optimization flags (e.g., `-Ofast`), there remains a possibility that specific, obscure preprocessor definitions or micro-architecture tuning flags required to fully unlock the DSP pipeline were not triggered. The "black box" nature of cross-compiling legacy C code within a modern Zig build infrastructure introduces variables that are difficult to trace without deep inspection of the generated assembly.

4) *The Depthwise Convolution Anomaly*: The most distinct outlier is the Depthwise Convolution (Cases 10-12 on Nicla, cases 9-14 on Nucleo-64), where CMSIS-NN is significantly faster in some cases but slower in others. In the cases where ZANT lags significantly (e.g., +52% latency on Nicla), the primary bottleneck is identified as the data layout mismatch. CMSIS-NN requires a specific channel-first memory layout for depthwise operations, whereas ONNX (and thus ZANT) uses channel-last. The mandatory runtime reordering of weights—despite being optimized—adds a purely serial overhead that cannot be parallelized, effectively creating a bottleneck before the accelerated kernel can even execute.

5) *CMSIS-NN Benchmark Baseline*: The benchmarks consulted compare the performance of CMSIS-NN convolutions with those of baselines that are excessively simplified or, sometimes, not even described in detail, leaving room for the hypothesis that Zant's functions, being optimized to exploit vector instructions and special cases of convolution, are sufficient to produce significantly better results than the baselines adopted in the benchmarks found on the several articles, thus reducing the distance with CMSIS-NN and, sometimes, even performing better than it. In the article [2],

for example, the `arm_conv` function of CMSIS-DSP, used for 1d-convolutions and generally applied to signal processing, is used as a baseline, therefore it is quite likely that the simple use of basic optimizations, even simpler than those adopted by Zant, can immediately alter the difference in performance expected and reported in the consulted articles, re-evaluating the native implementation compared to that of CMSIS-NN.

6) *Future Investigation and Profiling*: To definitively pinpoint the root causes—distinguishing between cache misses, pipeline stalls, or instruction count—a more granular analysis is required. The most effective strategy would involve using a hardware debugger (e.g., SEGGER J-Link with Ozone) to inspect the call stack and measure the exact Cycle Count (CYCCNT) per function. Currently, due to hardware availability constraints, this low-level profiling was not possible. However, these results serve as a valuable baseline, highlighting that "drop-in" hardware acceleration is not always a silver bullet and that memory dynamics often outweigh raw compute power in embedded systems.

7) *Possible Future Changes In How CMSIS-NN Functions Are Used*: Since CMSIS-NN's optimizations for 3x3 filter convolutions generally seem to yield higher performance gains than native convolutions, while performance tends to favor the Zant implementation when using 5x5 filters, one might consider converting the 5x5 filter application to a double 3x3 filter application, thus improving performance, reducing memory usage, and taking full advantage of the features provided by Arm. Replacing filters larger than 3 with multiple passes using 3x3 filters can also be generalized to filters larger than 5x5. As shown in [15], this change is easily applicable and does not change the final result in terms of correctness, but it does lead to a performance improvement.

REFERENCES

- [1] "Zant foundation," official Zant's repository.
- [2] N. S. Liangzhen Lai and V. Chandra, "Cmsis-nn: efficient neural network kernels for arm cortex-m cpus," *arXiv preprint*, vol. abs/1801.06601, 2018. [Online]. Available: <https://arxiv.org/pdf/1801.06601>
- [3] A. Maciá-Lillo, S. Barrachina, G. Fabregat, and M. F. Dolz, "Optimizing convolutions for deep learning inference on arm cortex-m processors," *IEEE Internet of Things Journal*, vol. 11, 2024. [Online]. Available: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=arnumber=10513367>
- [4] R. B. A. D. G. Fouad Sakr, Francesco Bellotti and J. Doyle, "Memory-efficient cmsis-nn with replacement strategy," pp. 299–303, 2021, available as PDF; memory-efficient strategies for CMSIS-NN on embedded devices. [Online]. Available: https://unige.iris.cineca.it/retrieve/e268c4ce-1f2f-a6b7-e053-3a05fe0adea1/Memory-Efficient_CMSIS-NN_with_Replacement_Strategy.pdf
- [5] M. G. Valentino Peluso and A. Calimera, "Arbitrary-precision convolutional neural networks on low-power iot processors," in *2019 27th IFIP/IEEE International Conference on Very Large Scale Integration (VLSI-SoC)*. IEEE, 2019, pp. 142–147.
- [6] A. Ltd., "Cmsis-nn software library documentation," <https://arm-software.github.io/CMSIS5/NN/html/index.html>, 2025, accessed : Dec.2025.
- [7] A. C. Manuele Rusci, Marco Fariselli and L. Benini, "Leveraging automated mixed-low-precision quantization for tiny edge microcontrollers," *arXiv preprint*, vol. abs/2008.05124, 2020. [Online]. Available: <https://arxiv.org/abs/2008.05124>

- [8] P. A. M. B. G. V. Novac PE, Boukli Hacene G, "Quantization and deployment of deep neural networks on microcontrollers," *Sensors(Basel)*, 2021, pMCID: PMC8122998, PMID: 33922868. [Online]. Available: <https://pubmed.ncbi.nlm.nih.gov/33922868/>
- [9] B. C. M. Z. M. T. A. H. H. A. Benoit Jacob, Skirmantas Kligys and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, 2018. [Online]. Available: https://openaccess.thecvf.com/content_cvpr2018/papers/Jacob_Quantization_and_Training_CVPR2018_paper.pdf
- [10] N. A. A. S. M. A. H. S. D. K. D. Hyunho Ahn, Tian Chen, "Performance characterization of using quantization for dnn inference on edge devices: extended version," *arXiv preprint*, vol. abs/2303.05016, 2023. [Online]. Available: <https://arxiv.org/abs/2303.05016>
- [11] M. M. V. S. Animesh Jain, Shoubhik Bhattacharya and E. I. Yida Wang, Amazon Web Services, "Efficient 8-bit quantization pipeline for neural network inference," *arXiv preprint*, vol. abs/2006.10226, 2020. [Online]. Available: <https://arxiv.org/pdf/2006.10226.pdf>
- [12] GitHub Issue 1026, "reorder f32->u8 much slower than s8," <https://github.com/quantization/reorder/issues/1026>, accessed: Dec. 2025.
- [13] I. Corporation, "Quantization overview and practical considerations," <https://intel.github.io/neural-compressor/latest/docs/source/3x/quantization.html>, accessed: Dec. 2025.
- [14] E. Mind, "int8 scalar quantization for efficient neural network inference," <https://www.emergentmind.com/topics/int8-scalar-quantization>, accessed: Dec. 2025.
- [15] K. Simonyan and A. Zisserman, "Using small convolutional filters for efficient neural networks," *arXiv preprint*, vol. abs/1409.1556, 2015. [Online]. Available: <https://arxiv.org/pdf/1409.1556.pdf>